

## QUESTION 1 EXPLANATION

To implement parallel row-by-row matrix multiplication I edited the user-space program. everything runs in user-space using fork, a single pipe, write/read, and wait.

The core idea was to compute each row of the result matrix  $C$  in a separate child process, and then pass that row back to the parent through a pipe together with the row index so the parent can assemble the final matrix in the correct order.

```
//Matrix initialization
int A[N][N];
int B[N][N];
int C[N][N];

//Computing the row results
void compute_row(int row, int rowstore[N] ){
    for(int i = 0; i < N; i++){
        int sum = 0;
        for(int j = 0; j < N; j++){
            sum += A[row][j] * B[j][i];
        }
        rowstore[i] = sum;
    }
}

int main() {
```

I Used the matrices as global fixed-size arrays so each child inherits them after fork().

A simple function computes one row of  $C$  (row row) and writes results into the provided rowstore buffer. This helps to keep the child simple.

The parent creates one pipe and then spawns  $N$  children. Each child computes its assigned row  $i$ , writes the row index followed by the  $N$  ints for that row into the pipe, and exits. The parent closes the write end, reads  $N$  pairs (row idx, row\_data) and stores them into  $C[i][j]$ .

Child side (inside the fork loop):

```

int p[2]; //defining pipe
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++) {
        A[i][j] = i + 1;
        B[i][j] = j + 1;
    }
if(pipe(p) < 0){
    printf("pipe error");//-1 if pipe not created
    exit(0);
}
for(int i = 0; i<N; i++){
    int pid = fork(); // parent has the pid of the child >0 and child has pid 0
    if(pid < 0){
        printf("fork error");
        exit(0);
    }
    else if (pid ==0){ // if child process then pid is zero
        int rowx[N];
        close(p[0]); // closing read end
        compute_row(i, rowx);
        write(p[1], &i, sizeof(int)); row index
        write(p[1], rowx, sizeof(rowx)); // values in row
        close(p[1]);
        exit(0);
    }
}

```

Parent side (collecting results):

```

close(p[1]);
for (int i = 0; i < N; i++) {
    int row_idx;
    int row[N];
    read(p[0], &row_idx, sizeof(int)); // row index
    read(p[0], row, sizeof(row));
    for (int j = 0; j < N; j++) {
        C[row_idx][j] = row[j];
    }
}

close(p[0]);

```

Notes on ordering: children may write to the pipe in any order. Passing the row\_idx lets the parent place each row correctly in C regardless of arrival order.

## QUESTION 4 EXPLANATION

```
char history[HISTORY_SIZE][CMD_MAX];
int history_count = 0;
int history_index = 0;
// Simple strncpy substitute (xv6 doesn't have one)
void my_strncpy(char *dst, const char *src, int n) {
    while(n-- > 0 && *src)
        *dst++ = *src++;
    *dst = '\0';
}
// Store a command in history
void add_history(const char *cmd) {
    my_strncpy(history[history_count % HISTORY_SIZE], cmd, CMD_MAX);
    history_count++;
}
```

These declarations provide a circular buffer of history\_size commands, each up to cmd- bytes. Count counts total commands ever added (used to compute where to write next) and index is used for navigating recalled entries.

Strncpy copies the commands into history buffer, add\_history implements circular buffer.

```
// Get previous command
char* get_prev_history() {
    if (history_count == 0)
        return 0;
    history_index--;
    if (history_index < 0)
        history_index = 0;
    return history[history_index % HISTORY_SIZE];
}

// Reset index when new command is typed
void reset_history_index() {
    history_index = history_count;
}
```

get\_prev\_history() decrements history\_index and returns the corresponding stored string.

Reset\_History\_indec.This is to be called whenever the user starts a new line.

I replaced the standard getcmd to read input character by character and handle the recall/backspace behavior. Key parts:

- Prompt and reset:

```
int getcmd(char *buf, int nbuf) {
    printf("$ ");
    int i = 0;
    reset_history_index();
```

Character loop: if user presses newline, accept the line; if '!' is typed, fetch the previous command and replace the current input buffer with it; if backspace (127 or '\b') is typed, erase; otherwise append the typed character to the buffer and echo it:

```
while (1) {
    char c;
    if (read(0, &c, 1) < 1)
        return -1;

    if (c == '\n') {
        buf[i] = '\0';
        printf("\n");
        return 0;
    } else if (c == '!') { // go back
        char *prev = get_prev_history();
        if (prev) {
            // Clear current line
            while (i > 0) {
                write(1, "\b \b", 3);
                i--;
            }
            // Copy and show previous command
            my_strncpy(buf, prev, nbuf - 1);
            i = strlen(buf);
            write(1, buf, i);
        }
    } else if (c == 127 || c == '\b') { // backspace
        if (i > 0) {
```

For saving saving commands into history (in main()): After the shell trims leading whitespace and ensures the command is non-empty, the code calls `add_history(cmd)` to store it.